

## [Bài đọc] Quy tắc định dạng dữ liệu trả về

Trong phát triển API, dữ liệu trả về đóng vai trò như một “ngôn ngữ chung” giữa hệ thống và người dùng cuối. Cách dữ liệu được định dạng sẽ quyết định API có dễ sử dụng, dễ bảo trì và dễ mở rộng hay không. Vì vậy, việc tuân thủ các quy tắc định dạng dữ liệu trả về là một phần quan trọng trong việc thiết kế API hiện đại

### 1. Vai trò của dữ liệu trả về

- Dữ liệu trả về không chỉ đơn thuần là kết quả phản hồi từ server mà còn là cầu nối quan trọng giúp client hiểu được trạng thái xử lý của API. Nếu được tổ chức đồng nhất và rõ ràng, dữ liệu trả về sẽ:
  - Giúp lập trình viên dễ dàng phân tích và tích hợp vào ứng dụng
  - Giảm thiểu lỗi khi giao tiếp giữa client và server
  - Tăng hiệu quả phát triển phần mềm và cải thiện trải nghiệm người dùng

### 2. XML và JSON trong API

- Trong lịch sử phát triển web service, **XML** từng là định dạng phổ biến. Tuy nhiên, sự xuất hiện của **RESTful API** đã đưa **JSON** trở thành tiêu chuẩn nhờ vào nhiều ưu điểm nổi bật:
  - **Nhẹ và đơn giản**: JSON có cú pháp ngắn gọn, ít rườm rà hơn XML
  - **Tương thích cao**: JSON được hỗ trợ tốt trong hầu hết các ngôn ngữ lập trình hiện đại như JavaScript, Java, Python
  - **Hiệu quả**: JSON giúp việc truyền tải và xử lý dữ liệu nhanh chóng, đặc biệt phù hợp với ứng dụng web và di động
  - **Dễ đọc – dễ viết**: Dữ liệu trong JSON gần gũi với cấu trúc đối tượng trong lập trình, giúp dễ dàng ánh xạ vào code
- Với những ưu điểm này, JSON hiện nay trở thành định dạng phổ biến nhất trong thiết kế API

### 3. Cấu trúc dữ liệu trả về chuẩn

- Để API hoạt động hiệu quả, dữ liệu trả về cần có cấu trúc thống nhất. Một cấu trúc chuẩn thường bao gồm:



```
{
  "success": true,
  "message": "Lấy dữ liệu thành công",
  "data": { ... },
  "statusCode": 200
}
```

- **success (boolean)**: Biểu thị kết quả thành công hay thất bại
- **message (string)**: Cung cấp thông tin bổ sung, có thể là thông báo lỗi hoặc trạng thái xử lý
- **data (object/array)**: Chứa dữ liệu chính mà client cần
- **statusCode (number)**: Phản ánh mã trạng thái HTTP (200, 400, 404, 500, ...)
- Cấu trúc này giúp API trở nên rõ ràng, dễ hiểu và dễ dàng kiểm soát khi xử lý thành công hoặc lỗi

### 4. Xử lý dữ liệu lỗi

- Trong thực tế, không phải lúc nào API cũng trả về kết quả thành công. Khi xảy ra lỗi, cần có cách xử lý nhất quán để client dễ dàng nhận biết và khắc phục
- Ví dụ:



```
{
  "success": false,
  "message": "Người dùng không tồn tại",
  "data": null,
  "statusCode": 404
}
```

- Việc tách bạch dữ liệu thành công và dữ liệu lỗi, hoặc đưa chúng về cùng một cấu trúc có trường báo lỗi rõ ràng, sẽ tăng tính minh bạch và giúp người dùng API dễ dàng xử lý hơn

## 5. Nguyên tắc thiết kế định dạng dữ liệu trả về

- Để API trở nên chuyên nghiệp và dễ sử dụng, các nhà phát triển cần tuân thủ một số nguyên tắc cơ bản:
  - **Đồng nhất:** Sử dụng cùng một cấu trúc trả về cho toàn bộ API
  - **Dễ hiểu:** Đặt tên trường rõ ràng, tránh viết tắt hoặc gây mơ hồ
  - **Dễ bảo trì - mở rộng:** Cấu trúc đủ linh hoạt để có thể thêm trường mới trong tương lai mà không phá vỡ API cũ
  - **Tuân thủ chuẩn RESTful:** Sử dụng đúng mã trạng thái HTTP kết hợp với thông tin bổ sung trong body phản hồi

## 6. Tầm quan trọng của việc tuân thủ quy tắc định dạng

- Nếu mỗi API có cách trả về dữ liệu khác nhau, lập trình viên sẽ gặp khó khăn trong việc tích hợp và bảo trì. Ngược lại, một API có định dạng dữ liệu nhất quán sẽ:
  - Thân thiện với người dùng
  - Giúp giảm thiểu lỗi và tăng tính ổn định
  - Đảm bảo khả năng mở rộng và phát triển lâu dài

## 7. Kết luận

- Quy tắc định dạng dữ liệu trả về là một trong những nền tảng quan trọng nhất khi thiết kế API. Bằng cách áp dụng các nguyên tắc về **sự đồng nhất, rõ ràng, chuẩn hóa và khả năng mở rộng**, các nhà phát triển có thể xây dựng API không chỉ hoạt động hiệu quả mà còn thân thiện, bền vững và dễ sử dụng